



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Claude Cowork Masterclass

Course Code: C1382

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	3 August 2026	Initial aligned release of the slide deck, Learner Guide, Lesson Plan, and three connected Northstar Operations labs.	Course Trainer

TABLE OF CONTENTS

Introduction.....	4
Course Learning Outcomes.....	4
Before You Start — Preparation.....	4
Topic 01 — Getting Started with Claude Cowork.....	5
Chat Assistance and Agentic Delegation.....	6
The Cowork Task Lifecycle.....	6
Anatomy of a High-Signal Task Brief.....	7
Authority Should Match Consequence.....	7
Monitor, Review, and Recover.....	8
Lab 1 — Delegate Your First Task to Claude Cowork.....	8
Recap — Getting Started with Claude Cowork.....	16
Topic 02 — Automating Everyday Work with Cowork.....	17
From Messy Inputs to a Decision-Ready Pack.....	17
Safe File-Organisation Patterns.....	17
One Fact Base, Three Reading Experiences.....	18
Spreadsheet Analysis as a Reconciliation Loop.....	18
Synthesis with Traceable Evidence.....	19
Orchestrating a Multi-Step Task.....	19
Lab 2 — Turn a Messy Folder into a Management-Ready Report.....	20
Recap — Automating Everyday Work with Cowork.....	31
Topic 03 — Advanced Cowork Workflows.....	31
Choose the Most Precise Trusted Tool.....	32
A Playbook Is an Operational Contract.....	32
Parallelism Needs Dependency Control.....	33
Human Gates in a Cowork Workflow.....	33
From Personal Workflow to Team Practice.....	34
Guardrails Across the Workflow.....	34
Lab 3 — Build a Repeatable Cowork Workflow for Your Own Work.....	35
Recap — Advanced Cowork Workflows.....	48
Wrap-Up - Your Cowork Operating System.....	48
Next Steps.....	49
Glossary.....	49

Introduction

This Learner Guide is a self-contained study and practice companion for Claude Cowork Masterclass (C1382). It follows the same three-topic sequence as the slide deck and Lesson Plan, then expands every framework into detailed explanations and three connected labs. The course uses a fictional Northstar Retail Operations scenario so learners can practise without exposing real business or personal data.

The guide treats Cowork as an agentic workspace rather than a chat feature. You will learn to define a task contract, control access and authority, monitor a multi-step run, reconcile evidence across files, create professional outputs, and convert a successful session into a repeatable playbook. Product capabilities and interface labels can evolve, so use the latest Claude application and follow the concept and evidence requirements even if a label moves.

Course Learning Outcomes

- LO1: Explain Claude Cowork's agentic operating model, execution surfaces, connected-folder boundary, task lifecycle, permissions, and the human responsibilities that remain.
- LO2: Set up a Cowork project, write an outcome-based task brief, run and monitor a bounded session, review its evidence, and recover from common setup or scope problems.
- LO3: Use Cowork to organise synthetic files, analyse spreadsheet data, synthesise research and notes, and produce consistent document, spreadsheet, and presentation outputs with traceable sources.
- LO4: Design and run a multi-step workflow that separates parallel workstreams from dependencies, uses integrations deliberately, and places human approval gates before consequential writes or sharing.
- LO5: Convert a successful session into a reusable playbook with parameters, project instructions, quality checks, exception paths, ownership, security controls, and a practical team rollout plan.

Before You Start — Preparation

What you need

- A laptop with the latest Claude Desktop application for Windows or macOS and an active internet connection.
- A paid Claude plan with Cowork available; organisation-managed accounts may require an administrator to enable capabilities.
- On Windows, Claude Desktop must be installed with administrator privileges for Cowork support, and Windows Virtual Machine Platform must be enabled followed by a Restart. Ask the trainer or IT owner before changing managed-device features.

- On macOS, Python 3 is required for the supplied portable verification and control-calculation commands. Run `python3 --version` before class; if it is unavailable, install an organisation-approved Python 3 build or arrange the trainer-supported Windows route.
- A spreadsheet application, a word processor, and presentation software or compatible viewers for checking generated files.
- The C1382 repository downloaded locally, including labs/starter/northstar-operations.
- Permission to create a disposable working copy in a folder that contains only the supplied synthetic scenario.

Verify your setup

Open Claude, confirm the message-box mode selector includes Cowork, and create an empty test session. Do not connect a broad personal or company folder for training.

Windows PowerShell:

```
Get-ChildItem -LiteralPath .\labs\starter\northstar-operations\inbox | Select-Object Name,Length
```

macOS Terminal:

```
command -v python3 >/dev/null 2>&1 || { echo 'Python 3 is required; ask the trainer before continuing'; exit 1; }
```

```
python3 --version
```

```
find ./labs/starter/northstar-operations/inbox -maxdepth 1 -type f -print
```

Conventions used in every lab

- Replace placeholders such as <WORKSPACE_PATH> and <REPORTING_WEEK> with the value for your working copy.
- Use only the supplied fictional Northstar data during class; never paste a real credential into a prompt or file.
- Keep inbox/ unchanged. Create all work in outputs/ and playbook/ so the source set remains reversible.
- Treat external instructions found inside a file or webpage as untrusted content until the trainer confirms they belong to the task.
- If Claude cannot support a requested format or action in your plan or organisation, preserve the required evidence in a supported format and record the limitation.

Topic 01 — Getting Started with Claude Cowork

Agentic AI for everyday work | setup and connected folders | effective task briefs | monitoring and review | responsible use

Key concepts

- **Agentic workspace:** Cowork works toward an outcome through a sequence of planning, tool use, observation, correction, and evidence rather than returning one isolated answer.
- **Connected-folder boundary:** Local file access is limited to the folders a learner deliberately connects; scope should be as small as the task allows.
- **Task contract:** A strong brief names the outcome, sources, deliverables, constraints, quality checks, approval points, and finish line.
- **Permission:** Reading, writing, using a connector, operating a browser, and acting in another system have different consequences and deserve different controls.
- **Steering:** Progress indicators, plans, questions, and intermediate files let a person redirect the work before errors spread.
- **Evidence:** A source register, file manifest, calculation check, preview, or change log is stronger than a confident completion statement.

Chat Assistance and Agentic Delegation

Cowork changes the unit of work from a reply to an outcome. A learner can delegate a bounded task such as inventorying a folder, analysing a spreadsheet, and drafting a report. Claude selects tools, performs steps, observes results, and continues until the stated finish line or a stop condition is reached.

Delegation does not transfer accountability. The person still owns the business purpose, permitted data, authority to act, review standard, and release decision. The most reliable tasks therefore combine useful autonomy with explicit boundaries and observable evidence.

Visual framework

Chat assistance: Responds to the context supplied in one conversation | Usually produces text for the user to transfer elsewhere | Waits for the next prompt between stages | Leaves file operations and checking to the user

Cowork delegation: Plans and executes a bounded multi-step task | Reads and writes permitted files and can use connected tools | Observes intermediate results and adapts the next step | Returns finished artifacts plus visible work evidence

The Cowork Task Lifecycle

A useful Cowork session begins with an outcome and a boundary. Claude inspects the available context, proposes a route, uses permitted tools, and compares intermediate results with the requested quality checks. Errors and missing information are observations that should change the plan, not details to conceal.

The finish line must be testable. Examples include a manifest that accounts for every source file, a workbook whose totals match a control calculation, or a report whose factual claims name their source. A stop condition is equally important: missing authority,

sensitive data, an unclear destination, or a consequential external action should return control to the user.

Visual framework

- Receive the outcome and boundaries
- Inspect permitted context
- Plan the work and identify questions
- Use tools and create intermediate results
- Review evidence, correct, and deliver

Anatomy of a High-Signal Task Brief

Prompt quality is task design quality. An adjective such as 'professional' is too elastic on its own; a stronger brief names the audience, sections, length, file format, source period, calculation rules, and review checklist. This reduces avoidable interpretation while leaving Claude room to solve the task.

Separate facts, preferences, and permissions. Facts come from named sources. Preferences cover tone and layout. Permissions define what Claude may read or change. When these categories are mixed, a stylistic suggestion can be misread as authority to alter data or publish an output.

Visual framework

- Outcome — Describe the business result and intended audience, not only the activity.
- Sources — Name the allowed folders, files, links, and systems; exclude everything else.
- Deliverables — Specify filenames, formats, structure, location, and presentation standard.
- Constraints — State data rules, non-goals, protected files, time range, and assumptions.
- Approval gates — Identify which writes, revisions, or external actions require confirmation.
- Evidence — Define control totals, citations, previews, logs, or other observable checks.

Authority Should Match Consequence

Risk depends on both access and action. Reading a narrow synthetic folder is materially different from browsing a broad drive, and producing a draft is different from sending it. Start with the smallest scope and the most reversible output that can prove value.

Human review should concentrate at consequence boundaries. A manager may allow automatic calculations in a working copy but require confirmation before source files are replaced, a document is shared, or a conclusion becomes an operational decision. These gates preserve speed without making authority ambiguous.

Visual framework

Lower-consequence work: Read a synthetic training folder | Create a draft in a new output directory | Summarise supplied notes with citations | Calculate metrics that a person will verify

Higher-consequence work: Overwrite or remove source files | Send messages or share files externally | Use financial, personal, legal, or confidential data | Make commitments, purchases, access changes, or final decisions

Monitor, Review, and Recover

Monitoring is not continuous micromanagement. It means checking the points where a misunderstanding would become expensive: the proposed source set, an inferred calculation rule, the first artifact, and any action outside the local workspace. A short correction at the plan stage is cheaper than rebuilding a complete output.

Recovery begins from evidence. If an output is wrong, identify the failing claim, calculation, source selection, or format rule; provide the observed mismatch and the expected state; then request a focused revision. Do not restart a long task with the same vague brief and hope for a different result.

Visual framework

- Read the plan
- Watch scope and tool use
- Inspect intermediate evidence
- Steer with precise feedback
- Verify the final state

Lab 1 — Delegate Your First Task to Claude Cowork

Learning outcome: LO1 and LO2: explain the Cowork task lifecycle and permission boundary, create a narrow project, brief a bounded file-inventory task, monitor its execution, and verify the result against the source folder.

Goal: You create a personal working copy of the fictional Northstar Retail Operations folder, establish an immutable source baseline, and connect only that narrow copy to a new Cowork project. You then use the DELEGATE framework to ask Claude for a complete file inventory, review its plan before execution, watch the session for scope drift, and compare the finished manifest with the actual source files.

Duration: 75 minutes.

What you'll build

A protected synthetic workspace, reviewed project instructions, a complete file inventory, and a verification record. (Tools: Claude Cowork, Claude Desktop, local file access, File Explorer or Finder, PowerShell or Terminal, supplied Northstar Operations starter.)

Prerequisites

- The latest Claude Desktop application is installed and the Cowork mode is available on your paid plan.
- On Windows, install Claude Desktop with administrator privileges for Cowork support, enable Windows Virtual Machine Platform, and Restart the computer. Ask the trainer or IT owner before changing a managed device.
- On macOS, Python 3 is installed and `python3 --version` succeeds; the supplied hashing and reconciliation commands require it.
- The C1382 repository is available locally and contains `labs/starter/northstar-operations`.
- You can create a new local folder outside the course repository for a disposable working copy.
- Use only the supplied fictional Northstar data; do not connect a personal, customer, finance, human-resources, or company drive folder.

Step-by-step

1. From the C1382 repository root, create a new sibling working copy named `C1382-northstar-workspace`. Stop if the destination already exists so an earlier learner workspace is never overwritten. Move into the new copy and create the Lab 1 output folder.

Windows PowerShell:

```
$source = (Resolve-Path .\labs\starter\northstar-operations).Path

$target = Join-Path (Split-Path (Resolve-Path .).Path -Parent) 'C1382-northstar-workspace'

if (Test-Path -LiteralPath $target) { throw "Destination exists: $target" }

Copy-Item -Recurse -LiteralPath $source -Destination $target

Set-Location -LiteralPath $target

New-Item -ItemType Directory -Force .\outputs\lab-01 | Out-Null
```

macOS Terminal:

```
command -v python3 >/dev/null 2>&1 || { echo 'Python 3 is required; ask the
trainer before continuing'; exit 1; }

python3 --version

source_dir="$(pwd)/labs/starter/northstar-operations"

target_dir="$(dirname "$(pwd)")/C1382-northstar-workspace"
```

```
test ! -e "$target_dir" || { echo "Destination exists: $target_dir"; exit 1; }

cp -R "$source_dir" "$target_dir"

cd "$target_dir"

mkdir -p outputs/lab-01
```

2. Read 00_READ_ME_FIRST.md and management-brief.md. Confirm aloud that the scenario is fictional, inbox is the protected source set, outputs is the only writable area, and the Week 32 control values will be used later. Do not open Cowork until this boundary is clear.

Open these files in a text editor:

00_READ_ME_FIRST.md

management-brief.md

3. Create a baseline manifest for the inbox before Claude receives access. Record each filename, byte length, and SHA-256 hash in outputs/lab-01/00-inbox-baseline.csv, then count the source files. Keep this file for the final comparison.

Windows PowerShell:

```
Get-ChildItem -LiteralPath .\inbox -File | Sort-Object Name | ForEach-Object {

    $hash = (Get-FileHash -Algorithm SHA256 -LiteralPath $_.FullName).Hash

    [pscustomobject]@{Name=$_.Name; Bytes=$_.Length; SHA256=$hash}

} | Export-Csv -NoTypeInformation -Encoding utf8 .\outputs\lab-01\00-inbox-
baseline.csv

(Import-Csv .\outputs\lab-01\00-inbox-baseline.csv).Count
```

macOS Terminal:

```
python3 - <<'PY'

from hashlib import sha256

from pathlib import Path

files = sorted((p for p in Path('inbox').iterdir() if p.is_file()), key=lambda p:
p.name)

lines = [f"{sha256(p.read_bytes()).hexdigest()} {p.as_posix()}\n" for p in files]

Path('outputs/lab-01/00-inbox-baseline.sha256').write_text(''.join(lines),
encoding='utf-8')

print(len(files))
```

PY

4. Open Claude Desktop and confirm it is current. Select Cowork in the message-box mode selector. In the left navigation choose Projects, choose the plus control, select Use an existing folder, and select only C1382-northstar-workspace. Name the project Northstar Operations Lab.

Project name: Northstar Operations Lab

Connected folder: <ABSOLUTE_PATH_TO_C1382-northstar-workspace>

5. Add the following durable project instructions. Read the saved version back and correct any line that is missing. These are stable boundaries for all three labs, not the task brief for one run.

This project contains fictional training data for C1382. Treat inbox/ as read-only. Create or revise files only inside outputs/ and playbook/. Never remove, rename, move, or overwrite a source file. Never send, share, publish, upload, or connect another system without explicit approval in the current task. Separate source evidence, inference, and unresolved questions. Label unsupported business claims OWNER TO VERIFY. Before reporting completion, list created files and the checks performed.

6. Start a new task inside the project. Paste the task brief below. It requires a plan before work and limits the first task to inventory; do not ask Claude to organise, analyse, or write a management report yet.

Outcome: Create a reliable inventory of the protected inbox for an operations manager.

Sources: Read 00_READ_ME_FIRST.md, management-brief.md, and every file directly inside inbox/. Do not read outside this connected project.

Deliverables: Create outputs/lab-01/01-file-inventory.md with a table containing filename, type, reporting period, apparent purpose, key fields or sections, likely owner, sensitivity note, and any ambiguity. Create outputs/lab-01/01-file-inventory.csv with the same source rows and these columns: filename, type, reporting_period, apparent_purpose, key_fields_or_sections, likely_owner, sensitivity_note, ambiguity. End the Markdown file with source count, likely duplicates, missing expected inputs, and questions for an owner.

Constraints: Treat inbox/ as read-only. Do not rename, move, remove, overwrite, reorganise, or summarise the business results. Do not use external sources. Put uncertain classifications in an Unknown or OWNER TO VERIFY state instead of guessing.

Process: First show your plan, intended reads and two intended writes. Wait for my approval. After approval, execute the plan.

Evidence: Account for every direct inbox file exactly once in each inventory, name both created files, and state how you checked completeness.

7. Review Claude's plan against the task contract. It should read only the named local files and write the Markdown and CSV inventories in outputs/lab-01. If the plan includes source changes, broad folder access, external research, or another destination, steer it with the correction below. Approve only when the route matches the boundary.

Revise the plan to remain inside the connected C1382-northstar-workspace. Read the named sources only. Leave inbox/ unchanged. Create only outputs/lab-01/01-file-inventory.md and outputs/lab-01/01-file-inventory.csv. Treat ambiguity as OWNER TO VERIFY. Show the corrected read set and write set, then wait again.

8. Allow the bounded plan to run. Watch the progress indicators, filenames, and tool requests. If Claude asks for another folder, an external connection, a deletion, or a write outside outputs/lab-01, deny it and restate the boundary. When the task finishes, open the actual inventory file rather than relying on the completion message.

Expected output paths:

outputs/lab-01/01-file-inventory.md

outputs/lab-01/01-file-inventory.csv

9. Compare the machine-readable inventory with the source directory as a multiset. Continue only when the filename sets and counts match exactly, every source appears once, no extra or duplicate row exists, and the Markdown table agrees with the CSV. Ask for a focused revision if any check fails.

Windows PowerShell:

```
$sourceNames = @(Get-ChildItem -LiteralPath .\inbox -File | Sort-Object Name |
Select-Object -ExpandProperty Name)

$inventoryNames = @((Import-Csv .\outputs\lab-01\01-file-inventory.csv) | ForEach-
Object { $_.filename })

$duplicates = @($inventoryNames | Group-Object | Where-Object Count -ne 1)

$diff = @(Compare-Object ($sourceNames | Sort-Object) ($inventoryNames | Sort-
Object))

if ($duplicates -or $diff -or $sourceNames.Count -ne $inventoryNames.Count) {

    $duplicates | Format-Table Name,Count
```

```

$diff | Format-Table -AutoSize

throw 'Inventory is not an exact one-row-per-source match'

}

$markdown = Get-Content -LiteralPath .\outputs\lab-01\01-file-inventory.md -Raw

$badMentions = @($sourceNames | Where-Object { ([regex]::Matches($markdown,
[regex]::Escape($_))).Count -ne 1 })

if ($badMentions) { $badMentions; throw 'Markdown inventory does not mention every
source exactly once' }

'PASS exact filename multiset match; no duplicates or extras'

```

macOS Terminal:

```

python3 - <<'PY'

from collections import Counter

from pathlib import Path

import csv

source = sorted(p.name for p in Path('inbox').iterdir() if p.is_file())

with open('outputs/lab-01/01-file-inventory.csv', newline='', encoding='utf-8-
sig') as f:

    rows = list(csv.DictReader(f))

inventory = [row['filename'] for row in rows]

if Counter(source) != Counter(inventory) or len(source) != len(inventory):

    raise SystemExit(f'FAIL source={Counter(source)}
inventory={Counter(inventory)}')

markdown = Path('outputs/lab-01/01-file-inventory.md').read_text(encoding='utf-8')

bad = [name for name in source if markdown.count(name) != 1]

if bad:

    raise SystemExit(f'FAIL Markdown occurrences: {bad}')

print('PASS exact filename multiset match; no duplicates or extras')

PY

```

10. Recalculate the inbox hashes and compare them with the baseline. Create outputs/lab-01/01-verification.md with the project name, connected folder, source count, inventory path, filename check, baseline comparison, any denied requests, and the final result. Use a truthful note if your operating system used the alternative hash file.

Windows PowerShell:

```
$before = Import-Csv .\outputs\lab-01\00-inbox-baseline.csv

$after = Get-ChildItem -LiteralPath .\inbox -File | Sort-Object Name | ForEach-Object {

    [pscustomobject]@{Name=$_ .Name; Bytes=$_ .Length; SHA256=(Get-FileHash -Algorithm SHA256 -LiteralPath $_.FullName).Hash}

}

$diff = Compare-Object $before $after -Property Name,Bytes,SHA256

@"

# Lab 1 Verification

- Project: Northstar Operations Lab

- Connected folder: $((Resolve-Path .).Path)

- Source count: $($after.Count)

- Inventories: outputs/lab-01/01-file-inventory.md and outputs/lab-01/01-file-inventory.csv

- Exact filename multiset match in both inventories: <YES_OR_DETAILS>

- Inbox hash comparison: $(if ($diff) {'CHANGED - REVIEW REQUIRED'} else {'UNCHANGED'})

- Denied or corrected scope requests: <NONE_OR_DETAILS>

- Result: <VERIFIED_OR_ACTION_REQUIRED>

"@ | Set-Content -Encoding utf8 .\outputs\lab-01\01-verification.md

if ($diff) { $diff | Format-Table -AutoSize } else { 'Inbox unchanged' }
```

macOS Terminal:

```
shasum -a 256 -c outputs/lab-01/00-inbox-baseline.sha256
```

Then create outputs/lab-01/01-verification.md with the same fields in a text editor.

11. Close the loop in Cowork. Ask Claude to compare the task contract with the finished state and report only the deliverable, checks, limitations, and remaining owner questions. Add any useful omission to the verification record without changing inbox.

Compare the original task brief with the current workspace. Do not create or change a file. Report: (1) the deliverable path, (2) completeness evidence, (3) boundary evidence, (4) unresolved OWNER TO VERIFY items, and (5) one improvement to the next task brief.

Test it

Open outputs/lab-01/01-file-inventory.md and 01-file-inventory.csv and confirm both contain one row for every direct inbox file and no duplicate or extra rows. Confirm the Markdown file contains the expected columns, source count, duplicate observations, missing-input observations, and owner questions. Run the executable filename-multiset comparison and require PASS. Run the hash comparison and confirm the protected inbox is unchanged. Open outputs/lab-01/01-verification.md and confirm it records the project, connected folder, source count, both inventory paths, exact filename result, hash result, scope interventions, and final verified state.

Checkpoint for the next lab

Keep the connected C1382-northstar-workspace, unchanged inbox, reviewed project instructions, 00-inbox-baseline file, 01-file-inventory.md, 01-file-inventory.csv, and 01-verification.md. Lab 2 uses this complete verified source boundary to build the management-ready pack.

Troubleshooting

- Cowork is not visible: Confirm you are signed in to a paid plan, update Claude Desktop, restart the application, and ask the organisation owner whether Cowork is enabled. Use the web surface only if the trainer confirms it provides the required capability for this lab.
- Cowork is unavailable on Windows or reports a virtualization error: Confirm Claude Desktop was installed with administrator privileges. In an elevated PowerShell window, run `Get-WindowsOptionalFeature -Online -FeatureName VirtualMachinePlatform`. If the approved IT procedure permits it and the feature is disabled, run `Enable-WindowsOptionalFeature -Online -FeatureName VirtualMachinePlatform -All`, then use Restart rather than shut down. On a managed device, ask IT to make the change; do not bypass policy.
- python3 is unavailable on macOS: Stop before creating the baseline. Install an organisation-approved Python 3 build and confirm `python3 --version`, or use the trainer-supported Windows route. Do not substitute a weaker hash or skip the executable controls.

- Projects or Use an existing folder is not visible: Update Claude Desktop. If the project feature is unavailable on the current surface, start a Cowork task on desktop, connect only the working-copy folder, paste the same instructions at the start, and record the limitation in 01-verification.md.
- The folder selector points at the course repository: Cancel and select the sibling C1382-northstar-workspace only. The course repository contains build sources that are outside the lab's data boundary.
- Claude proposes moving or renaming files: Do not approve the plan. Restate that Lab 1 is inventory-only, inbox is protected, and the only permitted writes are outputs/lab-01/01-file-inventory.md and outputs/lab-01/01-file-inventory.csv.
- A filename is absent from the inventory: Provide the exact missing filename and ask Claude for a focused inventory revision. Rerun the complete filename comparison after the revision.
- The hash comparison reports a change: Stop. Identify the changed source file from the comparison, restore the working copy from labs/starter/northstar-operations, regenerate the baseline, and rerun Lab 1. Never copy a changed training source back into the course repository.

Challenge

Extend 01-verification.md with a file-type count derived from 01-file-inventory.csv, then compare that count with the Markdown summary. Explain any Unknown classification without changing a source file.

Reflection

Which part of the task contract prevented the largest possible scope error, and which observable check proved that the boundary held?

Note: Full commands and screenshots are in labs/lab-01-*.md. Use only the supplied fictional data. Keep inbox/ unchanged, create outputs in new folders, verify important values, and obtain clear authority before any external write, sharing action, source replacement, or removal.

Recap — Getting Started with Claude Cowork

You can now:

- LO1 and LO2: explain the Cowork task lifecycle and permission boundary, create a narrow project, brief a bounded file-inventory task, monitor its execution, and verify the result against the source folder

Carry forward the verified lab checkpoints and resolve any OWNER TO VERIFY items with the named owner before the next topic.

Topic 02 — Automating Everyday Work with Cowork

File organisation | documents, reports and presentations | spreadsheet analysis | research and summarisation | end-to-end multi-step tasks

Key concepts

- Inventory before organisation: A manifest reveals what exists, duplicates, uncertain items, and protected sources before any move or rename is proposed.
- Working-copy pattern: Transform copies into a new destination so originals remain a stable reference and recovery path.
- Structured extraction: Convert documents and notes into fields, evidence, assumptions, and open questions before writing a narrative.
- Calculation contract: Define each metric's numerator, denominator, period, unit, rounding, and control total before interpreting it.
- Artifact system: A workbook, report, and presentation should share one fact base while serving different reading behaviours.
- Provenance: Important claims remain traceable to a filename, row, note, or authoritative external source.

From Messy Inputs to a Decision-Ready Pack

Multi-file work becomes reliable when it is treated as a pipeline. Inventory establishes the source boundary. Extraction separates observed facts from assumptions. Calculation turns raw rows into defined metrics. Composition produces artifacts for different audiences. Review reconciles every output to the same evidence.

The stages are connected but should not be collapsed. If Claude drafts a polished narrative before reconciling the numbers, presentation quality can hide a factual error. A staged workflow makes intermediate artifacts inspectable and gives the learner a safe place to correct the process.

Visual framework

- Inventory and classify inputs
- Extract facts and unresolved questions
- Calculate and reconcile metrics
- Compose audience-specific artifacts
- Review, revise, and package evidence

Safe File-Organisation Patterns

Organisation is an information-governance task, not cosmetic tidying. A useful structure reflects how work is retrieved, owned, retained, and reviewed. An Unknown category is safer than a confident but unsupported classification.

Reversibility is the governing design principle. The first run should create a manifest and an organised working copy, not silently rename or remove originals. The source folder

remains the control set until a person approves the proposed structure and confirms that every input is accounted for.

Visual framework

- Manifest — Record filename, type, date, purpose, likely owner, sensitivity, and proposed destination.
- Classification — Use stable business categories and an explicit Unknown bucket instead of guessing.
- Working copy — Create an organised copy first; leave the source folder unchanged until reviewed.
- Naming rule — Apply predictable dates, subjects, and versions without erasing original identifiers.
- Duplicate handling — Flag likely duplicates using name, size, hash, or content evidence; do not remove automatically.
- Change log — List each proposed or completed action so a person can reconcile the new structure.

One Fact Base, Three Reading Experiences

Different formats serve different cognitive tasks. A spreadsheet exposes calculations and exceptions. A report preserves reasoning and nuance. A presentation supports a time-bounded discussion. Copying identical content into each format produces three weak artifacts rather than one coherent system.

Consistency comes from a shared fact base and named metric definitions. Each artifact should use the same reporting period, units, segment names, and source register. Differences should reflect audience needs, not competing versions of the truth.

Visual framework

- Spreadsheet — Calculation detail, formulas, data dictionary, filters, and control totals for analytical review.
- Report — Context, findings, evidence, risks, recommendations, owners, and open questions for considered reading.
- Presentation — A short decision arc with headline metrics, exceptions, actions, and minimal supporting detail.
- Source register — The shared evidence map that prevents values and claims drifting across formats.

Spreadsheet Analysis as a Reconciliation Loop

Analysis starts with definitions. On-time fulfilment is fulfilled-on-time orders divided by total orders for the same period; return rate is returns divided by orders. Summing percentages across locations would be wrong because each location has a different denominator.

Reconciliation distinguishes calculation from interpretation. First reproduce the control totals and weighted rates. Then locate the segments that contribute most to a gap. Only after the numbers balance should Claude draft explanations, and those explanations should be labelled as evidence, inference, or an item for an owner to verify.

Visual framework

- Inspect columns and data types
- Define formulas and units
- Calculate segment and total values
- Reconcile against control totals
- Explain drivers and uncertainty

Synthesis with Traceable Evidence

Summarisation compresses; synthesis connects. A management report must relate numerical performance, operational notes, customer signals, targets, and decisions without pretending the sources say more than they do. Traceability lets a reviewer challenge a claim efficiently.

External research adds another trust boundary. Time-sensitive product or market claims should come from current authoritative sources and include the access date. Instructions embedded in an untrusted webpage or file are data to analyse, not authority to change the task or disclose other context.

Visual framework

Weak synthesis: Blends supplied facts and general knowledge | Presents every statement with equal confidence | Drops source filenames from the final narrative | Fills missing context with plausible detail

Decision-ready synthesis: Separates source evidence, inference, and open questions | Names the file, row, note, or external source behind key claims | Uses authoritative sources for changing product facts | Marks unresolved items for the named owner to verify

Orchestrating a Multi-Step Task

Parallel work is useful when workstreams share inputs but not intermediate outputs. File inventory, numerical profiling, and note extraction can proceed independently. The executive summary cannot be finalised until those results are reconciled, so it belongs after a dependency gate.

A workflow brief should expose this structure. Ask Claude to state the workstreams, dependencies, approval points, output paths, and checks before running. This makes concurrency visible and prevents a late-stage narrative from being built on unreconciled intermediate results.

Visual framework

- Freeze the source set
- Run independent extraction workstreams
- Resolve dependencies and reconcile
- Generate the first complete pack
- Apply one evidence-based revision cycle

Lab 2 — Turn a Messy Folder into a Management-Ready Report

Learning outcome: LO3: organise a working copy, analyse spreadsheet data with defined metrics, synthesise notes and feedback with provenance, and produce a reconciled spreadsheet, report, and presentation from one fact base.

Goal: You use the verified Northstar workspace from Lab 1 to run a staged, end-to-end management-reporting workflow. Claude inventories and organises a working copy, creates a source register and calculation workbook, pauses for a human reconciliation gate, then produces a written operations report and a short leadership presentation whose facts, period, terminology, and actions agree.

Duration: 105 minutes.

What you'll build

A reconciled Week 32 workbook, source and evidence registers, management report, leadership brief, quality review, and run log. (Tools: Claude Cowork, local files, spreadsheet creation and analysis, document creation, presentation creation, spreadsheet and office viewers.)

Prerequisites

- Lab 1 is complete or its rejoin baseline has been reconstructed and verified.
- `outputs/lab-01/01-file-inventory.md` accounts for every direct inbox file.
- `outputs/lab-01/01-file-inventory.csv` contains the same exact one-row-per-source filename set.
- The current inbox hashes match `outputs/lab-01/00-inbox-baseline.csv` or its macOS equivalent.
- The Northstar Operations Lab project instructions still restrict writes to `outputs/` and `playbook/`.
- On macOS, `python3 --version` succeeds; the independent controls and revision-hash records require Python 3.

Step-by-step

1. Open `C1382-northstar-workspace` and verify the Lab 1 checkpoint. Confirm the inventory and verification files exist, the verification state is clear, and the protected inbox still matches the baseline. Create the new output directory only after the check succeeds.

Windows PowerShell:

```

$required = @('.\outputs\lab-01\00-inbox-baseline.csv','.\outputs\lab-01\01-file-
inventory.md','.\outputs\lab-01\01-file-inventory.csv','.\outputs\lab-01\01-
verification.md')

$required | ForEach-Object { if (-not (Test-Path -LiteralPath $_)) { throw
"Missing checkpoint: $_" } }

$verification = Get-Content -LiteralPath .\outputs\lab-01\01-verification.md -Raw

if ($verification -notmatch 'Result:\s*VERIFIED') { throw 'Lab 1 verification is
not in a verified state' }

$before = Import-Csv .\outputs\lab-01\00-inbox-baseline.csv

$after = Get-ChildItem .\inbox -File | Sort-Object Name | ForEach-Object
{ [pscustomobject]@{Name=$_.Name;Bytes=$_.Length;SHA256=(Get-FileHash -Algorithm
SHA256 $_.FullName).Hash} }

if (Compare-Object $before $after -Property Name,Bytes,SHA256) { throw 'Inbox
changed; restore the verified checkpoint' }

New-Item -ItemType Directory -Force .\outputs\lab-02 | Out-Null

```

macOS Terminal:

```
set -euo pipefail
```

```
command -v python3 >/dev/null 2>&1 || { echo 'Python 3 is required; ask the
trainer before continuing'; exit 1; }
```

```
python3 --version
```

```
for f in outputs/lab-01/00-inbox-baseline.sha256 outputs/lab-01/01-file-
inventory.md outputs/lab-01/01-file-inventory.csv outputs/lab-01/01-
verification.md; do test -f "$f" || { echo "Missing $f"; exit 1; }; done
```

```
grep -Eq 'Result:[[:space:]]*VERIFIED' outputs/lab-01/01-verification.md || { echo
'Lab 1 is not verified'; exit 1; }
```

```
shasum -a 256 -c outputs/lab-01/00-inbox-baseline.sha256
```

```
mkdir -p outputs/lab-02
```

105-minute phase budget: checkpoint and controls 15; Stage 1 execution 25; reconciliation gate 15; Stage 2 generation 25; visual review and revisions 20; final close-out 5. If Stage 1 has not reconciled by minute 55, compare your calculations with checkpoints/lab-02-stage1-control.md, correct the failing calculation or source mapping, and continue only after the human gate passes.

2. Read management-brief.md and brand-style.md. In outputs/lab-02/02-quality-review.md, create headings for source scope, metric reconciliation, workbook review, report review, presentation review, cross-artifact consistency, inbox comparison, and owner questions. This file is your independent review record, not Claude's completion summary.

Create this file in a text editor:

outputs/lab-02/02-quality-review.md

Required headings:

```
# Lab 2 Quality Review

## Source scope

## Metric reconciliation

## Workbook review

## Report review

## Presentation review

## Cross-artifact consistency

## Inbox comparison

## Owner questions

## Final result
```

3. Calculate the Week 32 control totals directly from northstar-sales-w32.csv. Record the values in the Metric reconciliation section before Claude creates the workbook. Use weighted rates from the total numerators and denominators, not the average of location percentages.

Windows PowerShell:

```
$rows = Import-Csv .\inbox\northstar-sales-w32.csv

$orders = ($rows | Measure-Object orders -Sum).Sum

$onTime = ($rows | Measure-Object fulfilled_on_time -Sum).Sum

$revenue = ($rows | Measure-Object revenue_sgd -Sum).Sum

$returns = ($rows | Measure-Object returns -Sum).Sum

$hours = ($rows | Measure-Object labour_hours -Sum).Sum

[pscustomobject]@{Orders=$orders;OnTime=$onTime;RevenueSGD=$revenue;Returns=$returns;LabourHours=$hours;OnTimeRate=''{0:P2}'' -f
```

```
($onTime/$orders));ReturnRate=('{0:P2}' -f
($returns/$orders));RevenuePerOrder=('{0:N2}' -f ($revenue/$orders))} | Format-
List
```

```
$rows | Where-Object { [double]$_.fulfilled_on_time / [double]$_.orders -lt 0.95 }
| ForEach-Object { '{0}: {1:P2}' -f
$_.location,([double]$_.fulfilled_on_time/[double]$_.orders) }
```

macOS Terminal:

```
python3 - <<'PY'
```

```
import csv
```

```
with open('inbox/northstar-sales-w32.csv', newline='', encoding='utf-8-sig') as f:
```

```
    rows = list(csv.DictReader(f))
```

```
def total(name): return sum(float(r[name]) for r in rows)
```

```
orders, on_time = total('orders'), total('fulfilled_on_time')
```

```
revenue, returns = total('revenue_sgd'), total('returns')
```

```
hours = total('labour_hours')
```

```
print(f'Orders = {orders:.0f}')
```

```
print(f'Fulfilled on time = {on_time:.0f}')
```

```
print(f'Revenue = SGD {revenue:.0f}')
```

```
print(f'Returns = {returns:.0f}')
```

```
print(f'Labour hours = {hours:.0f}')
```

```
print(f'On-time fulfilment = {on_time/orders:.2%}')
```

```
print(f'Return rate = {returns/orders:.2%}')
```

```
print(f'Revenue per order = SGD {revenue/orders:.2f}')
```

```
print('Exceptions below 95%:')
```

```
for row in rows:
```

```
    rate = float(row['fulfilled_on_time']) / float(row['orders'])
```

```
    if rate < 0.95: print(f"{row['location']}: {rate:.2%}")
```

```
PY
```

Expected controls:

Orders = 2000

Fulfilled on time = 1856

Revenue = SGD 339350

Returns = 113

Labour hours = 2733

On-time fulfilment = 92.80%

Return rate = 5.65%

Revenue per order = SGD 169.68

Exceptions below 95% = Orchard 94.05%, Jurong 90.41%, and Woodlands 87.30%

- 4. Return to the Northstar Operations Lab project and start a new Cowork task.**
Paste the staged brief below. It requires three independent extraction workstreams and a pause after the workbook so the narrative cannot outrun reconciled evidence.

Outcome: Turn the protected Week 32 Northstar inputs into a management-ready operating pack for the Head of Operations.

Approved sources: management-brief.md, brand-style.md, outputs/lab-01/01-file-inventory.md, inbox/northstar-sales-w32.csv, inbox/customer-feedback-w32.csv, and inbox/operations-notes-w32.md. These six paths are the complete read set. Do not read, analyse, cite, or copy any Week 33 file in this task. Do not use external sources.

Stage 1 workstreams that may run in parallel:

A. File workstream - create outputs/lab-02/organised-copy/ with copies of the three Week 32 inputs and the two stable brief/style files, classified into data, notes, feedback, and reference. Leave inbox unchanged. Create outputs/lab-02/02-source-register.md with source, period, purpose, use, and limitation.

B. Data workstream - create outputs/lab-02/02-operations-analysis.xlsx with sheets Read Me, Raw Sales, KPI Summary, Location Detail, Feedback, and Sources. Preserve raw rows. Use formulas where practical. Define every KPI and show control totals.

C. Evidence workstream - create outputs/lab-02/02-evidence-register.md with observation or theme, evidence type, exact source and location, confidence, inference, OWNER TO VERIFY item, and intended downstream use.

Required Week 32 KPIs: total orders, fulfilled-on-time orders, weighted on-time fulfilment rate, revenue, returns, weighted return rate, labour hours, revenue per order, and location exceptions against the targets in management-brief.md.

Human gate: First show your plan, exact read set, write set, workstream dependencies, and checks. Wait for approval. After approval, complete Stage 1 only. Pause and show the source register, evidence register, workbook paths, KPI values, exceptions, formula or calculation method, and unresolved questions. Do not draft the report or presentation until I approve Stage 2.

Stage 2 deliverables after later approval: outputs/lab-02/02-management-report.docx, outputs/lab-02/02-leadership-brief.pptx, and outputs/lab-02/02-run-log.md. Follow brand-style.md and management-brief.md. Do not send, share, publish, remove, or overwrite anything.

5. Review the proposed plan. Confirm that the three Stage 1 workstreams read the same frozen source set, write to distinct destinations, and merge through the source register and KPI summary. Confirm that report and presentation work wait for the metric gate. Correct the plan if any source write, external source, or premature narrative appears.

The approved route must show:

- only the three named Week 32 inputs and the stable brief/style files are content sources; Week 33 is excluded
- inbox/ is read-only and organised-copy/ contains copies only
- source and evidence registers have distinct destinations and jointly form shared evidence
- the workbook preserves raw rows and definitions
- Stage 1 pauses before the report and slides
- no send, share, publish, removal, or overwrite action

If needed, say: Revise the plan to match these six conditions, show the corrected read and write sets, and wait again.

6. Approve Stage 1 and let Claude execute. Monitor the paths and intervening questions. When Claude pauses, open 02-source-register.md, 02-evidence-register.md, and 02-operations-analysis.xlsx. Confirm the workbook contains the six promised sheets; the source register accounts for every file used; and the evidence register gives exact provenance, separates evidence from inference, and identifies

every OWNER TO VERIFY item. Do not approve Stage 2 from the completion message alone.

Expected Stage 1 paths:

outputs/lab-02/organised-copy/

outputs/lab-02/02-source-register.md

outputs/lab-02/02-evidence-register.md

outputs/lab-02/02-operations-analysis.xlsx

7. Reconcile the workbook with your independent controls. Check the reporting period, total orders, fulfilled-on-time orders, revenue, returns, labour hours, weighted rates, and revenue per order. Confirm Orchard, Jurong, and Woodlands are clearly identified as on-time exceptions and that the calculation does not average location percentages. Record the observed workbook cells and results in 02-quality-review.md.

Expected workbook values:

- Reporting period: Week 32
- Orders: 2,000
- Fulfilled on time: 1,856
- On-time fulfilment: 92.80%
- Revenue: SGD 339,350
- Returns: 113
- Return rate: 5.65%
- Labour hours: 2,733
- Revenue per order: SGD 169.68
- On-time exceptions: Orchard 94.05%, Jurong 90.41%, and Woodlands 87.30%

Record the actual sheet and cell for each value in outputs/lab-02/02-quality-review.md.

8. If a value, formula, sheet, source reference, or exception is wrong, give Claude a focused discrepancy report. Require correction in the workbook and source register only, then repeat the complete reconciliation. Approve Stage 2 only after every control matches or an unresolved item is explicitly labelled for an owner.

Focused correction template:

Stage 1 is not approved yet. In <FILE_AND_LOCATION>, I observed <ACTUAL>. The control is <EXPECTED> because <DEFINITION_OR_SOURCE>. Correct the root calculation or source mapping, preserve the raw rows, update any dependent Stage 1 view, and show the revised values and checks. Do not begin Stage 2.

9. Approve Stage 2 with the final evidence contract below. It requires the report and slides to use the reconciled workbook, cite the source register, distinguish fact from inference, and keep recommendations reviewable.

Stage 1 is approved. Complete Stage 2 using only the reconciled workbook, source register, and evidence register as the fact base.

Report: Create outputs/lab-02/02-management-report.docx with title, period, executive summary, KPI table, location exceptions, customer-feedback themes, evidence-based drivers, three prioritised recommendations, owner and timing fields, risks, OWNER TO VERIFY items, methodology, and source and evidence register references.

Presentation: Create outputs/lab-02/02-leadership-brief.pptx with 5 to 7 slides: title, executive scorecard, exceptions, customer and operational evidence, recommended actions, decisions and owner questions. Use concise headlines and the style rules in brand-style.md.

Run log: Create outputs/lab-02/02-run-log.md listing sources read, files created, metric definitions, control checks, corrections, assumptions, owner questions, and confirmation that inbox was unchanged.

Do not invent root causes, commitments, owners, dates, or external facts. Use OWNER TO VERIFY where the supplied sources do not resolve a claim. Do not send, share, or publish the artifacts.

10. Open the actual Word document and presentation in compatible applications. Review the complete report for structure, page breaks, clipped or blank content, metric consistency, evidence labels, and actionable recommendations. Review every slide for readable text, clean layout, consistent period and values, a decision arc, and no unsupported claim. Record specific findings in 02-quality-review.md.

Visual review checklist:

Report - title and period; complete sections; readable tables; no blank pages; no clipped text; sources and methodology; OWNER TO VERIFY items.

Slides - 5 to 7 slides; readable at normal presentation size; no overlaps; no cut-off text; same Week 32 values; clear decisions and owner questions.

Cross-artifact - same metric names, units, rounding, locations, targets, recommendations, and source period.

11. Before requesting a revision, record pre-revision hashes for every Stage 1 and Stage 2 artifact. Then give Claude one evidence-based revision request for every real defect. Name the file, location, observed issue, expected state, and invariant that must remain unchanged. After revision, reopen every changed artifact and repeat the full cross-artifact check.

Windows PowerShell:

```
$reviewFiles = Get-ChildItem .\outputs\lab-02 -File | Where-Object Name -Match
'^02-(?!pre-revision-hashes)'
```

```
$reviewFiles | Sort-Object Name | ForEach-Object
{ [pscustomobject]@{Name=$_ .Name;SHA256=(Get-FileHash -Algorithm SHA256
$_ .FullName).Hash} } | Export-Csv -NoTypeInformation -Encoding utf8 .
\outputs\lab-02\02-pre-revision-hashes.csv
```

macOS Terminal:

```
python3 - <<'PY'
```

```
from hashlib import sha256
```

```
from pathlib import Path
```

```
folder = Path('outputs/lab-02')
```

```
excluded = {'02-pre-revision-hashes.sha256', '02-post-revision-hashes.sha256'}
```

```
files = sorted((p for p in folder.glob('02-*') if p.is_file() and p.name not in
excluded), key=lambda p: p.name)
```

```
Path(folder/'02-pre-revision-
hashes.sha256').write_text(''.join(f"{sha256(p.read_bytes()).hexdigest()}
{p.as_posix()}\n" for p in files), encoding='utf-8')
```

```
print(f'Recorded {len(files)} pre-revision hashes')
```

```
PY
```

Revision template:

Revise <FILE>. At <PAGE_SLIDE_SHEET_OR_SECTION>, I observed <ISSUE>. Change it to <EXPECTED_STATE>. Preserve the verified Week 32 source set, control totals, metric definitions, source and evidence registers, and every other correct section. After saving, list the exact changes and rerun the cross-artifact consistency check.

12. Recalculate the inbox hashes against the Lab 1 baseline. Complete the Inbox comparison, Owner questions, and Final result sections in 02-quality-review.md. Ask Claude for a read-only final reconciliation across the workbook, report, presentation, source register, and run log, then compare its report with your own record.

Windows PowerShell:

```
$before = Import-Csv .\outputs\lab-01\00-inbox-baseline.csv

$after = Get-ChildItem .\inbox -File | Sort-Object Name | ForEach-Object
{ [pscustomobject]@{Name=$_.Name;Bytes=$_.Length;SHA256=(Get-FileHash -Algorithm
SHA256 $_.FullName).Hash} }

$diff = Compare-Object $before $after -Property Name,Bytes,SHA256

if ($diff) { $diff | Format-Table -AutoSize; throw 'Inbox changed' } else { 'Inbox
unchanged' }

$afterRevision = Get-ChildItem .\outputs\lab-02 -File | Where-Object Name -Match
'^02-(?!pre-revision-hashes)' | Sort-Object Name | ForEach-Object
{ [pscustomobject]@{Name=$_.Name;SHA256=(Get-FileHash -Algorithm SHA256
$_.FullName).Hash} }

$beforeRevision = Import-Csv .\outputs\lab-02\02-pre-revision-hashes.csv

Compare-Object $beforeRevision $afterRevision -Property Name,SHA256 | Format-Table
-AutoSize
```

macOS Terminal:

```
shasum -a 256 -c outputs/lab-01/00-inbox-baseline.sha256

python3 - <<'PY'

from hashlib import sha256

from pathlib import Path

folder = Path('outputs/lab-02')

excluded = {'02-pre-revision-hashes.sha256', '02-post-revision-hashes.sha256'}

files = sorted((p for p in folder.glob('02-*') if p.is_file() and p.name not in
excluded), key=lambda p: p.name)

Path(folder/'02-post-revision-
hashes.sha256').write_text(''.join(f"{sha256(p.read_bytes()).hexdigest()}
{p.as_posix()}\n" for p in files), encoding='utf-8')

print(f'Recorded {len(files)} post-revision hashes')

PY
```

```
diff -u outputs/lab-02/02-pre-revision-hashes.sha256 outputs/lab-02/02-post-revision-hashes.sha256 || true
```

Record which hashes changed and why in 02-quality-review.md. A requested revision should change only the intended artifacts.

Cowork prompt:

Read the current Lab 2 artifacts without changing them. Reconcile reporting period, KPI definitions, values, rounding, exception locations, recommendations, owner questions, source references, and file list across 02-operations-analysis.xlsx, 02-management-report.docx, 02-leadership-brief.pptx, 02-source-register.md, 02-evidence-register.md, and 02-run-log.md. Report every mismatch; if none, state the checks performed and any remaining limitation.

Test it

Confirm outputs/lab-02 contains organised-copy and the seven named artifacts: source register, evidence register, workbook, report, presentation, quality review, and run log. Confirm organised-copy contains only the named Week 32 inputs and stable brief/style files, with no Week 33 file. Reconcile the workbook to 2,000 orders, 1,856 fulfilled on time, SGD 339,350 revenue, 113 returns, 2,733 labour hours, 92.80% on-time fulfilment, 5.65% return rate, and SGD 169.68 revenue per order; confirm Orchard, Jurong, and Woodlands are the on-time exceptions. Open and visually inspect the report and every slide. Confirm period, values, definitions, exception locations, recommendations, and owner questions agree across the workbook, report, presentation, source register, evidence register, and run log. Confirm the inbox hash comparison is unchanged, the before/after hash record explains intended revisions, and 02-quality-review.md records every check and the final result.

Checkpoint for the next lab

Keep the verified Lab 2 package and unchanged inbox. Lab 3 derives a reusable weekly-operations playbook from this run, changes the reporting-week parameter to Week 33, and proves the workflow can repeat without copying Week 32 conclusions.

Troubleshooting

- Claude begins the report before the metric gate: Steer immediately: stop Stage 2, retain any draft as non-authoritative, reconcile Stage 1, then replace the draft from the approved workbook rather than editing unsupported claims in place.
- The workbook averages location rates: Require weighted totals: sum fulfilled_on_time divided by sum orders, and sum returns divided by sum orders. Preserve each location's rate for exception analysis but do not average those percentages for the overall rate.

- A generated file will not open: Ask Claude to regenerate that one artifact in the same required format and path from the reconciled fact base. Keep the source register and verified workbook unchanged, then reopen the replacement.
- The report invents a cause or owner: Replace the unsupported statement with an evidence-labelled inference or OWNER TO VERIFY item. Add the question and intended owner to the report and quality review.
- The slide deck is dense or clipped: Request a focused layout revision: shorten headlines and body copy, split one overloaded slide, preserve the decision arc and all verified values, then inspect every slide again.
- Values disagree across files: Treat the reconciled workbook as the numerical source of truth and the source register as the provenance source. Ask Claude to rebuild the dependent report and slide statements, then rerun the full cross-artifact check.

Challenge

Ask Claude to add an executive dashboard sheet with one trend-free Week 32 scorecard, target variance, conditional formatting, and a plain-language data dictionary. Verify that the dashboard does not imply a time trend from one week of data and that every value reconciles to the existing KPI Summary.

Reflection

Which intermediate artifact made the final pack most trustworthy, and why would generating all three formats in one unreviewed step have increased risk?

Note: Full commands and screenshots are in labs/lab-02-*.md. Use only the supplied fictional data. Keep inbox/ unchanged, create outputs in new folders, verify important values, and obtain clear authority before any external write, sharing action, source replacement, or removal.

Recap — Automating Everyday Work with Cowork

You can now:

- LO3: organise a working copy, analyse spreadsheet data with defined metrics, synthesise notes and feedback with provenance, and produce a reconciled spreadsheet, report, and presentation from one fact base

Carry forward the verified lab checkpoints and resolve any OWNER TO VERIFY items with the named owner before the next topic.

Topic 03 — Advanced Cowork Workflows

Integrations | repeatable playbooks | parallel work and longer projects | approvals and guardrails | team rollout

Key concepts

- Integration choice: Prefer the most precise trusted path: a connector for a cloud service, a desktop capability for local context, or a browser only when no better interface exists.
- Playbook: A reusable workflow describes triggers, parameters, sources, stages, deliverables, checks, gates, exceptions, and ownership.
- Project instructions: Durable local context stores stable conventions and boundaries; task-specific details remain in the individual brief.
- Parallel workstream: Independent stages can run together, while dependent stages wait for reconciled inputs.
- Human-in-the-loop: A named person reviews evidence and explicitly authorises consequential action at a defined gate.
- Operational rollout: A team needs approved use cases, data boundaries, ownership, monitoring, incident handling, and a feedback cycle.

Choose the Most Precise Trusted Tool

A tool is not interchangeable with another merely because both can reach the same system. Structured connectors are usually more precise for cloud data. Desktop capabilities are appropriate for local files or applications. Browser and screen interaction are fallback paths when no direct interface exists.

Integration design begins with trust and scope: who owns the connection, which records it exposes, which actions it allows, where results are stored, and how access is removed. Convenience does not answer these governance questions.

Visual framework

Structured connection: Connector returns defined records from a cloud service |
Authentication and scope are explicit | Faster and less sensitive to interface changes |
Prefer for repeatable cloud workflows when available

Screen interaction: Browser or computer use navigates the visible interface | Can reach tools without a direct connection | Slower and more vulnerable to layout or state changes |
Use cautiously and review before consequential actions

A Playbook Is an Operational Contract

A saved prompt is not yet a playbook. Repeatability requires variable parameters, a stable source contract, defined intermediate artifacts, observable quality checks, and a response when an input is missing or contradictory. Ownership turns the document from advice into an operating procedure.

The playbook should remain tool-aware without becoming a transcript of one successful run. Describe the intended capability and preferred path, then identify a safe fallback. This makes the workflow resilient when an interface or connection changes.

Visual framework

- Trigger and parameters — Define when the workflow starts and which period, team, folder, or audience changes each run.
- Source contract — Name required inputs, freshness, owner, fallback, and the condition that stops the run.
- Stages and dependencies — Separate parallel extraction from reconciliation and dependent composition.
- Deliverables — Fix output names, formats, destinations, and minimum required sections.
- Quality gates — Attach calculations, source checks, visual review, and approval to named stages.
- Exceptions and ownership — State who resolves missing inputs, conflicting data, access failure, and release decisions.

Parallelism Needs Dependency Control

Parallel work reduces elapsed time only when workstreams are independent and their outputs have clear contracts. If two workstreams redefine the same metric or edit the same artifact, coordination cost can exceed the gain.

A longer project needs merge points. At each one, reconcile terminology, periods, totals, source references, and unresolved questions before dependent work continues. The merge point is also a natural place for a human review gate.

Visual framework

Can run in parallel: Profile a frozen spreadsheet source | Extract themes from supplied notes | Inventory files and build a source register | Draft format shells without final claims

Must wait: Reconcile metrics across workstreams | Choose the final management narrative | Approve recommendations and owners | Share, send, publish, or replace source material

Human Gates in a Cowork Workflow

A human gate is useful only when it names the decision, reviewer, evidence, and permitted next action. 'Check with me' is vague; 'pause after the workbook reconciles and show the totals, exceptions, and source register before drafting recommendations' is operational.

Not every stage needs a gate. Frequent low-value interruptions encourage reflexive approval. Place gates where access broadens, an assumption becomes a business conclusion, a source is changed, or an output leaves the working environment.

Visual framework

- Approve source scope

- Approve metric definitions
- Review the first complete draft
- Authorise consequential actions
- Record release evidence and owner

From Personal Workflow to Team Practice

A team rollout should begin with a narrow process whose inputs, outputs, owner, and current baseline are known. Synthetic or explicitly approved data lowers risk during learning. The pilot should measure both benefit and control cost: time saved, revision rate, factual corrections, missed exceptions, and reviewer effort.

Scaling means standardising the useful parts while preserving escalation. Teams need approved use cases, data classification, access provisioning, playbook ownership, versioning, monitoring, incident response, and a channel for lessons from real runs to improve the workflow.

Visual framework

- Choose a bounded low-consequence use case
- Define baseline time and quality
- Pilot with synthetic or approved data
- Review value, errors, and control load
- Standardise, train, monitor, and improve

Guardrails Across the Workflow

Guardrails are layered. A narrow folder reduces exposure; a working-copy rule reduces impact; a calculation check reduces factual error; a human gate reduces the chance that a draft becomes an unauthorised action. No single control covers every failure mode.

The controls must evolve with the workflow. New connectors, broader source sets, scheduled execution, computer use, or a change from drafting to acting all increase consequence and require a fresh review of permissions, monitoring, ownership, and recovery.

Visual framework

- Data — Use the minimum necessary approved sources and keep sensitive records outside training workspaces.
- Access — Grant the narrowest folder and connection scope; review and remove access when the workflow ends.
- Action — Default to drafts and working copies; gate external writes, deletion, sharing, and commitments.
- Evidence — Retain source registers, control totals, assumptions, reviewer decisions, and run logs.

- Quality — Use defined metrics, exception checks, visual inspection, and a named accountable reviewer.
- Change — Version playbooks and project instructions; retest after source, tool, or policy changes.

Lab 3 — Build a Repeatable Cowork Workflow for Your Own Work

Learning outcome: LO4 and LO5: convert a successful multi-step session into a parameterised playbook, choose tools and integrations deliberately, control parallel work with human gates, rerun the workflow on a new period, and define a safe team rollout.

Goal: You turn the verified Lab 2 run into a reusable weekly-operations playbook instead of saving one long prompt. The playbook defines parameters, source contracts, parallel workstreams, dependency gates, deliverables, control checks, exception paths, and ownership. You then change the reporting-week parameter to Week 33, run the workflow, reconcile the new outputs, and document whether local files, connectors, browser actions, or scheduled execution are appropriate.

Duration: 85 minutes.

What you'll build

A parameterised playbook, integration decision, verified Week 33 pack, quality record, and bounded rollout plan. (Tools: Claude Cowork projects, project instructions, local files, parallel workstreams, office artifact creation, optional trusted connectors, human review gates.)

Prerequisites

- Labs 1 and 2 are complete or the Lab 3 rejoin baseline has been reconstructed and verified.
- The protected inbox matches the Lab 1 baseline and the Lab 2 quality review has a clear final state.
- The Lab 2 workbook reconciles to the Week 32 controls and the report and presentation agree with it.
- Week 33 source files are present in inbox/ and have not been analysed or altered for this lab.
- On macOS, python3 --version succeeds; the independent controls and revision-hash records require Python 3.

Step-by-step

1. Verify the connected checkpoint before deriving a reusable process. Confirm the Lab 2 artifacts and quality review exist, read the final result and remaining owner questions, and recalculate the inbox hash comparison. Create the playbook and Week 33 output directories only after the source boundary is confirmed.

Windows PowerShell:

```
$required = @('.\outputs\lab-01\00-inbox-baseline.csv','.\outputs\lab-01\01-file-inventory.md','.\outputs\lab-01\01-file-inventory.csv','.\outputs\lab-01\01-verification.md','.\outputs\lab-02\02-source-register.md','.\outputs\lab-02\02-evidence-register.md','.\outputs\lab-02\02-operations-analysis.xlsx','.\outputs\lab-02\02-management-report.docx','.\outputs\lab-02\02-leadership-brief.pptx','.\outputs\lab-02\02-quality-review.md','.\outputs\lab-02\02-run-log.md','.\inbox\northstar-sales-w33.csv','.\inbox\customer-feedback-w33.csv','.\inbox\operations-notes-w33.md')
```

```
$required | ForEach-Object { if (-not (Test-Path -LiteralPath $_)) { throw "Missing checkpoint: $_" } }
```

```
$before = Import-Csv .\outputs\lab-01\00-inbox-baseline.csv
```

```
$after = Get-ChildItem .\inbox -File | Sort-Object Name | ForEach-Object { [pscustomobject]@{Name=$_.Name;Bytes=$_.Length;SHA256=(Get-FileHash -Algorithm SHA256 $_.FullName).Hash} }
```

```
if (Compare-Object $before $after -Property Name,Bytes,SHA256) { throw 'Inbox changed; restore the verified checkpoint' }
```

```
New-Item -ItemType Directory -Force .\playbook, .\outputs\lab-03\week-33 | Out-Null
```

macOS Terminal:

```
set -euo pipefail
```

```
command -v python3 >/dev/null 2>&1 || { echo 'Python 3 is required; ask the trainer before continuing'; exit 1; }
```

```
python3 --version
```

```
for f in outputs/lab-01/00-inbox-baseline.sha256 outputs/lab-01/01-file-inventory.md outputs/lab-01/01-file-inventory.csv outputs/lab-01/01-verification.md outputs/lab-02/02-source-register.md outputs/lab-02/02-evidence-register.md outputs/lab-02/02-operations-analysis.xlsx outputs/lab-02/02-management-report.docx outputs/lab-02/02-leadership-brief.pptx outputs/lab-02/02-quality-review.md outputs/lab-02/02-run-log.md inbox/northstar-sales-w33.csv inbox/customer-feedback-w33.csv inbox/operations-notes-w33.md; do test -f "$f" || { echo "Missing $f"; exit 1; }; done
```

```
shasum -a 256 -c outputs/lab-01/00-inbox-baseline.sha256
```

```
mkdir -p playbook outputs/lab-03/week-33
```

85-minute phase budget: checkpoint and playbook 18; integration and project rules 7; independent controls and Stage 1 15; reconciliation gate 7; Stage 2 artifact generation 13; artifact and visual review 10; rollout planning 7; final

reconciliation, hash review, release decision, and close-out 8. If the playbook is not approved by minute 20, adapt checkpoints/lab-03-playbook-starter.md. If Stage 1 has not reconciled by minute 47, use checkpoints/lab-03-week33-reconciliation.md to locate and correct the mismatch. If Stage 2 is incomplete by minute 62, use checkpoints/lab-03-stage2-review-checkpoint.md to focus the remaining review; no checkpoint replaces either human gate.

- 2. In the Northstar Operations Lab project, ask Claude to inspect the task brief, run log, quality review, source register, and final artifacts from Lab 2 without changing them. Require it to distinguish stable workflow rules from Week 32-specific parameters and conclusions.**

Read these Lab 2 artifacts without changing files: 02-source-register.md, 02-evidence-register.md, 02-quality-review.md, 02-run-log.md, the workbook structure and definitions, the report structure, and the presentation structure. Produce a response with two tables: (A) stable workflow elements that should repeat every week, and (B) Week 32-specific parameters, values, exceptions, and conclusions that must never be copied into a new run. Cite the artifact behind each item.

- 3. Ask Claude to draft the reusable workflow files below. The playbook must describe a process, not preserve Week 32 results. Project instructions must remain concise and durable. The quality checklist must contain observable checks and a named evidence location for each stage.**

Using the stable elements you identified, create three files.

1. `playbook/weekly-operations-review.md` with: purpose; owner; trigger; parameters REPORTING_WEEK, SOURCE_FOLDER, OUTPUT_FOLDER, AUDIENCE, and TARGET_SET; required-source contract; stop conditions; three parallel extraction workstreams; dependency and reconciliation gate; artifact-generation stage; draft-review gate; deliverables; metric definitions; quality checks; exception paths; run log; release owner; and version history.

2. `playbook/project-instructions.md` with no more than 12 concise rules covering synthetic or approved data, read-only sources, output boundaries, evidence versus inference, weighted rates, owner verification, human gates, no external action without approval, run logs, and final reconciliation.

3. `playbook/quality-checklist.md` as a table with stage, check, expected evidence, owner, and response when the check fails. Include source completeness, freshness, control totals, weighted rates, formula preservation, cross-artifact consistency, visual review, inbox comparison, unresolved questions, and release decision.

Do not copy Week 32 values, exception locations, or recommendations into reusable rules. Do not change the project settings yet.

- 4.** Review the three files. Search for Week 32 values, location conclusions, or recommendations that were accidentally hard-coded. Confirm the playbook contains every required parameter, gate, exception path, and owner. Revise the files before using them if any required element is missing or any old conclusion survives.

Windows PowerShell:

```
Select-String -Path .\playbook\*.md -Pattern '339,350|92.80|5.65|Woodlands|Jurong'
-CaseSensitive:$false
```

Expected result: no Week 32 value or exception is encoded as a reusable rule. A location name may appear only in an explicitly labelled example that cannot affect execution; remove it for this lab.

Manual playbook check:

- five parameters present
- source contract and stop conditions present
- parallel workstreams and dependency gate present
- reconciliation and draft-review gates present
- deliverables and metric definitions present
- exception paths, owner, run log, and version history present

- 5.** Create an integration decision record with Claude. Require one row each for local files, a remote connector, a desktop capability, browser interaction, computer use, and scheduled execution. For this lab, choose an on-demand local-file run.

Distinguish a remote built-in schedule, which cannot be tied to a computer folder, from a manually configured schedule that requires local files or apps and therefore runs only locally. Any future schedule requires an approved source, an available execution surface, and a fresh control review.

Create `playbook/integration-decision.md` as a decision table with capability, candidate use, data location, access scope, read or write actions, reliability, key risk, human gate, fallback, owner, and decision. Include local connected folder, remote connector, desktop capability, browser interaction, computer use, and scheduled task.

Decision for this training run: use the connected local working copy on demand; use no external connector, browser action, computer use, or scheduled task. Record

both scheduling modes: remote built-in schedules run without the computer and must use connectors or files saved to the Claude account, not a computer folder; a manually configured schedule that requires local files or apps runs locally and depends on the local device and required app being available. Before either mode is approved, review source access, freshness, availability, monitoring, failure handling, and release authority.

6. Compare playbook/project-instructions.md with the current project instructions. In the Cowork project settings, update the instructions only after reading the final text. Preserve the original local-source protections, and do not paste Week 32 values or the Week 33 task brief into durable instructions.

Project-instruction review:

- stable across reporting weeks
- no metric values or location conclusions
- no broad folder or connection authority
- explicit read-only source and output rules
- explicit evidence, owner-verification, and release gates

Then paste the reviewed contents of playbook/project-instructions.md into the Northstar Operations Lab project instructions and save.

7. Calculate Week 33 control totals independently from northstar-sales-w33.csv and record them in a new outputs/lab-03/03-week-33-quality-review.md before the run. This proves the playbook processes a new period instead of repeating Week 32 values.

Windows PowerShell:

```
$rows = Import-Csv .\inbox\northstar-sales-w33.csv

$orders = ($rows | Measure-Object orders -Sum).Sum

$onTime = ($rows | Measure-Object fulfilled_on_time -Sum).Sum

$revenue = ($rows | Measure-Object revenue_sgd -Sum).Sum

$returns = ($rows | Measure-Object returns -Sum).Sum

$hours = ($rows | Measure-Object labour_hours -Sum).Sum

$controls =
[pscustomobject]@{Orders=$orders;OnTime=$onTime;RevenueSGD=$revenue;Returns=$returns;LabourHours=$hours;OnTimeRate=('{0:P2}' -f ($onTime/$orders));ReturnRate=('{0:P2}' -f ($returns/$orders));RevenuePerOrder=('{0:N2}' -f ($revenue/$orders))}
```

```

$controls | Format-List

@"

# Week 33 Quality Review


## Independent controls

$( $controls | Format-List | Out-String)

## Source contract

## Reconciliation gate

## Draft-review gate

## Artifact and visual review

## Cross-artifact consistency

## Revision hash comparison

## Inbox comparison

## Exceptions and owner questions

## Release decision

"@ | Set-Content -Encoding utf8 .\outputs\lab-03\03-week-33-quality-review.md

```

macOS Terminal:

```

python3 - <<'PY'

from pathlib import Path

import csv

with open('inbox/northstar-sales-w33.csv', newline='', encoding='utf-8-sig') as f:

    rows = list(csv.DictReader(f))

def total(name): return sum(float(r[name]) for r in rows)

orders, on_time = total('orders'), total('fulfilled_on_time')

revenue, returns = total('revenue_sgd'), total('returns')

hours = total('labour_hours')

lines = [

```

```

'# Week 33 Quality Review', '', '## Independent controls',

f'- Orders: {orders:.0f}', f'- Fulfilled on time: {on_time:.0f}',

f'- Revenue: SGD {revenue:.0f}', f'- Returns: {returns:.0f}',

f'- Labour hours: {hours:.0f}', f'- On-time fulfilment: {on_time/orders:.2%}',

f'- Return rate: {returns/orders:.2%}', f'- Revenue per order: SGD
{revenue/orders:.2f}',

'', '### Exceptions below 95%',

]

lines += [f"- {r['location']}:
{float(r['fulfilled_on_time'])/float(r['orders']):.2%}" for r in rows if
float(r['fulfilled_on_time'])/float(r['orders']) < .95]

lines += ['', '## Source contract', '## Reconciliation gate', '## Draft-review
gate', '## Artifact and visual review', '## Cross-artifact consistency', '##
Revision hash comparison', '## Inbox comparison', '## Exceptions and owner
questions', '## Release decision']

Path('outputs/lab-03/03-week-33-quality-review.md').write_text('\n'.join(lines) +
'\n', encoding='utf-8')

print('\n'.join(lines[:15]))

PY

```

Expected controls:

Orders = 2100

Fulfilled on time = 1985

Revenue = SGD 358050

Returns = 101

Labour hours = 2777

On-time fulfilment = 94.52%

Return rate = 4.81%

Revenue per order = SGD 170.50

Exceptions below 95% = Woodlands 90.91% and Jurong 92.11%

- 8.** Start a new Cowork task and run the playbook with explicit Week 33 parameters. Require a plan and source-contract check first, then a pause at the reconciliation gate. The workstreams may run in parallel, but dependent narrative work must wait.

Run `playbook/weekly-operations-review.md` with these parameters:

`REPORTING_WEEK = Week 33`

`SOURCE_FOLDER = inbox`

`OUTPUT_FOLDER = outputs/lab-03/week-33`

`AUDIENCE = Head of Operations`

`TARGET_SET = the targets in management-brief.md`

Use only `inbox/northstar-sales-w33.csv`, `inbox/customer-feedback-w33.csv`, `inbox/operations-notes-w33.md`, `management-brief.md`, and `brand-style.md`. Do not use Week 32 values, exception locations, or recommendations as evidence for Week 33.

First show the parameter resolution, exact source set, write set, parallel workstreams, dependencies, stop conditions, two human gates, and quality checks. Wait for approval. After approval, complete Stage 1 and create only `source-register.md`, `evidence-register.md`, `operations-analysis.xlsx`, and `reconciliation.md` in the Week 33 output folder. Pause at the reconciliation gate with those four paths, the calculated controls, exceptions, and unresolved questions. Do not generate the report, slides, or run log until I approve the gate.

- 9.** Review and approve the plan only if it resolves the correct Week 33 files and paths. After extraction, compare Claude's reconciliation with the independent controls. Confirm the overall rates are weighted and the workflow identified Week 33 exceptions from Week 33 data. Record the checks, discrepancies, corrections, and approval decision in `03-week-33-quality-review.md`.

Required Stage 1 paths:

- `outputs/lab-03/week-33/source-register.md`
- `outputs/lab-03/week-33/evidence-register.md`
- `outputs/lab-03/week-33/operations-analysis.xlsx`
- `outputs/lab-03/week-33/reconciliation.md`

Expected Week 33 reconciliation:

- Orders: 2,100

- Fulfilled on time: 1,985
- Revenue: SGD 358,050
- Returns: 101
- Labour hours: 2,777
- Weighted on-time fulfilment: 94.52%
- Weighted return rate: 4.81%
- Revenue per order: SGD 170.50
- On-time exceptions against 95%: Woodlands 90.91% and Jurong 92.11%

If any path or value differs, do not approve the gate. Give the observed and expected state, metric definition, and source file; require a focused correction and rerun all controls.

10. Approve Stage 2 only after the reconciliation gate. The source register, evidence register, workbook, and reconciliation already exist from Stage 1; Stage 2 creates only the report, presentation, and run log. Require a second pause at the draft-review gate before any release decision.

Reconciliation gate approved. Continue the playbook and create only these remaining Week 33 artifacts inside outputs/lab-03/week-33:

- management-report.docx
- leadership-brief.pptx
- run-log.md

Use the four approved Stage 1 artifacts as the reconciled Week 33 fact base. Preserve the playbook's structures, definitions, checks, and owner fields, but derive all values, exceptions, customer themes, and recommendations from Week 33 sources. In run-log.md, record the exact Stage 1 gate evidence and decision. Pause at the draft-review gate. Do not share, send, publish, replace a source, or mark the pack released.

11. At the draft-review gate, open the workbook, document, and every slide. Run the quality checklist: file presence, source completeness, formulas and controls, metric definitions, period labels, evidence versus inference, owner questions, layout, cross-artifact consistency, and inbox comparison. Before any revision, record hashes for all seven Week 33 artifacts. Record exact locations for defects and request focused revisions. Repeat every affected check after a change. After the reviewer makes the draft-review decision, update the run log with the actual second-gate evidence before final reconciliation.

Windows PowerShell:

```
Get-ChildItem .\outputs\lab-03\week-33 -File | Sort-Object Name | ForEach-Object {  
[pscustomobject]@{Name=$_Name;SHA256=(Get-FileHash -Algorithm SHA256  
$_FullName).Hash} } | Export-Csv -NoTypeInformation -Encoding utf8 .  
\outputs\lab-03\03-pre-revision-hashes.csv
```

macOS Terminal:

```
python3 - <<'PY'
```

```
from hashlib import sha256
```

```
from pathlib import Path
```

```
folder = Path('outputs/lab-03/week-33')
```

```
files = sorted((p for p in folder.iterdir() if p.is_file()), key=lambda p: p.name)
```

```
Path('outputs/lab-03/03-pre-revision-  
hashes.sha256').write_text(''.join(f"{sha256(p.read_bytes()).hexdigest()}  
{p.as_posix()}\n" for p in files), encoding='utf-8')
```

```
print(f'Recorded {len(files)} pre-revision hashes')
```

```
PY
```

Artifact review:

- all seven named files exist: source and evidence registers, reconciliation, workbook, report, presentation, and run log
- every artifact says Week 33
- workbook and reconciliation agree with the independent controls
- report and slides use 94.52%, 4.81%, SGD 358,050, and the same rounding
- no Week 32 value or conclusion appears as Week 33 evidence
- source register names the Week 33 inputs
- unsupported claims are labelled OWNER TO VERIFY
- report has no blank or clipped page
- slides have no overlap, cut-off text, or unreadable density

After all revisions and repeated checks, record the draft-review decision in 03-week-33-quality-review.md. Then say:

Update only outputs/lab-03/week-33/run-log.md. Record the actual draft-review gate reviewer, evidence examined, corrections, decision, date and time, and precisely permitted next action. Do not invent approval, share a file, or change another artifact. Reopen run-log.md and verify this second-gate record before continuing.

12. Ask Claude for a read-only final reconciliation. Compare its evidence with your quality review, then make the release decision for the local training pack. The decision authorises only retaining the files in the workspace; it does not authorise sharing or any external action.

Read the Week 33 artifacts, playbook, quality checklist, and current quality review without changing them. Reconcile the source set, period, metric definitions, values, rounding, exceptions, recommendations, owner questions, file list, gate decisions, and run log. Report mismatches and limitations. If none remain, state the evidence for local training-pack readiness. Do not share, send, publish, or change a file.

13. Create the rollout plan with Claude, then edit it to your own work context without naming real data or credentials. Keep the first pilot draft-only and bounded. Include a baseline, owner, approved source class, access process, evidence, review burden, monitoring, incident response, and stop conditions.

Create outputs/lab-03/03-rollout-plan.md for a four-week pilot of one recurring knowledge-work process. Include: use-case boundary and non-goals; accountable owner and reviewers; permitted data classes and prohibited data; source and access provisioning; task and playbook owner; current time and quality baseline; draft-only Week 1 run; success and guardrail metrics; human gates; run evidence; exception and incident response; access removal; weekly review; version control; criteria to continue, narrow, pause, or stop. Use placeholders rather than real names, systems, data, or credentials.

14. Run the final inbox hash comparison and inspect the project for all promised outputs. Complete the quality review's Inbox comparison, Exceptions and owner questions, and Release decision sections. Record one playbook improvement discovered during the Week 33 run and update the playbook version history to v1.1 only if you actually make that improvement.

Windows PowerShell:

```
$before = Import-Csv .\outputs\lab-01\00-inbox-baseline.csv

$after = Get-ChildItem .\inbox -File | Sort-Object Name | ForEach-Object
{ [pscustomobject]@{Name=$_ .Name;Bytes=$_ .Length;SHA256=(Get-FileHash -Algorithm
SHA256 $_.FullName).Hash} }

if (Compare-Object $before $after -Property Name,Bytes,SHA256) { throw 'Inbox
changed' } else { 'Inbox unchanged' }
```

```

$postRevision = Get-ChildItem .\outputs\lab-03\week-33 -File | Sort-Object Name |
ForEach-Object { [pscustomobject]@{Name=$_.Name;SHA256=(Get-FileHash -Algorithm
SHA256 $_.FullName).Hash} }

$postRevision | Export-Csv -NoTypeInformation -Encoding utf8 .\outputs\lab-03\03-
post-revision-hashes.csv

Compare-Object (Import-Csv .\outputs\lab-03\03-pre-revision-hashes.csv)
$postRevision -Property Name,SHA256 | Format-Table -AutoSize

Get-ChildItem .\playbook -File | Select-Object Name,Length

Get-ChildItem .\outputs\lab-03\week-33 -File | Select-Object Name,Length

if (-not (Test-Path .\outputs\lab-03\03-week-33-quality-review.md)) { throw
'Missing quality review' }

if (-not (Test-Path .\outputs\lab-03\03-rollout-plan.md)) { throw 'Missing rollout
plan' }

```

macOS Terminal:

```

set -euo pipefail

shasum -a 256 -c outputs/lab-01/00-inbox-baseline.sha256

python3 - <<'PY'

from hashlib import sha256

from pathlib import Path

folder = Path('outputs/lab-03/week-33')

files = sorted((p for p in folder.iterdir() if p.is_file()), key=lambda p: p.name)

Path('outputs/lab-03/03-post-revision-
hashes.sha256').write_text(''.join(f"{sha256(p.read_bytes()).hexdigest()}
{p.as_posix()}\n" for p in files), encoding='utf-8')

print(f'Recorded {len(files)} post-revision hashes')

PY

diff -u outputs/lab-03/03-pre-revision-hashes.sha256 outputs/lab-03/03-post-
revision-hashes.sha256 || true

find playbook outputs/lab-03 -maxdepth 2 -type f -print

test -f outputs/lab-03/03-week-33-quality-review.md

test -f outputs/lab-03/03-rollout-plan.md

```

Record which artifact hashes changed and why in the Revision hash comparison section. Only intended revision targets should differ. Confirm run-log.md contains the actual second-gate reviewer, evidence, decision, time, and permitted next action before making the release decision.

Test it

Confirm the four playbook files contain the required parameters, source contract, workstreams, dependencies, two human gates, deliverables, controls, exception paths, ownership, and tool decision, with no Week 32 conclusion hard-coded. Confirm Stage 1 produced source-register.md, evidence-register.md, operations-analysis.xlsx, and reconciliation.md before approval, then Stage 2 produced management-report.docx, leadership-brief.pptx, and run-log.md. Reconcile the seven Week 33 artifacts to 2,100 orders, 1,985 fulfilled on time, SGD 358,050 revenue, 101 returns, 2,777 labour hours, 94.52% on-time fulfilment, 4.81% return rate, and SGD 170.50 revenue per order. Open and visually inspect the report and every slide, confirm the registers, reconciliation, and run log describe Week 33, confirm both human gates are recorded, confirm the pre/post revision hash comparison explains only intended changes, confirm the inbox is unchanged, and confirm the rollout plan names the bounded use case, data rules, owner, baseline, evidence, monitoring, incident response, and stop criteria.

Checkpoint for the next lab

Retain the verified local Week 33 pack, playbook files, quality review, and rollout plan as the final C1382 learning portfolio. Before applying the workflow to real work, obtain the relevant data and system approvals, substitute an explicitly approved source contract, and rerun every control in a draft-only pilot.

Troubleshooting

- The playbook contains Week 32 values or recommendations: Remove results from the reusable procedure. Replace them with parameters, definitions, target references, and instructions for deriving the new period's exceptions.
- Parallel workstreams produce conflicting definitions: Stop at the dependency gate. Select the definitions in the playbook, reconcile the upstream outputs, update the shared source register and control sheet, then restart dependent work.
- Week 33 output repeats a Week 32 conclusion: Treat it as contamination from prior context. Name the copied claim, require Claude to locate Week 33 evidence, remove unsupported carry-over, and rerun the complete reconciliation across every artifact.
- A scheduled run cannot access its source: Keep this lab on demand while diagnosing the execution mode. A remote built-in schedule cannot be tied to a computer folder, so use an approved connector or a file saved to the Claude account. A manually configured schedule that requires local files or apps runs locally, so confirm the device and required app are available. Re-review access, freshness, monitoring, failure handling, and release authority before enabling either route.

- A connector is unavailable or not approved: Use the local synthetic source for this lab. Record the connector as Not approved or Not available in integration-decision.md and keep the workflow's source contract independent of that tool.
- The project instructions become a long task prompt: Keep only durable rules that apply across reporting weeks. Move parameters, sources, audience, and one-run deliverables back into the task invocation.
- The run log omits a human gate: Do not declare the local pack ready. Add the actual reviewer, evidence examined, decision, time, and permitted next action; never fabricate an approval that did not occur.

Challenge

If the trainer provides an approved connector to a synthetic cloud folder, clone the playbook as a connector variant. Compare source resolution, access scope, evidence, latency, failure mode, and fallback with the local run. Do not add a real business connection and do not enable a schedule during class.

Reflection

Which element transformed the successful Lab 2 session into an operational playbook, and where did the Week 33 run reveal that a saved prompt alone would not have been enough?

Note: Full commands and screenshots are in labs/lab-03-*.md. Use only the supplied fictional data. Keep inbox/ unchanged, create outputs in new folders, verify important values, and obtain clear authority before any external write, sharing action, source replacement, or removal.

Recap — Advanced Cowork Workflows

You can now:

- LO4 and LO5: convert a successful multi-step session into a parameterised playbook, choose tools and integrations deliberately, control parallel work with human gates, rerun the workflow on a new period, and define a safe team rollout

Carry forward the verified lab checkpoints and resolve any OWNER TO VERIFY items with the named owner before the next topic.

Wrap-Up - Your Cowork Operating System

The three labs form one operating system for responsible delegation: a task contract, a reconciled artifact pipeline, and a repeatable playbook with human control.

Before every run

- Confirm the outcome, owner, approved source scope, deliverables, and evidence.
- Choose the most precise trusted tool and the smallest reversible action.
- Name the stop conditions and approval gates before work begins.

Before every release

- Reconcile control totals, metric definitions, reporting periods, and source references.
- Inspect the actual document, spreadsheet, and presentation rather than trusting filenames.
- Record unresolved claims, the accountable reviewer, the release decision, and any follow-up action.

Next Steps

- Repeat all three labs from a fresh working copy and compare the evidence, not just the visual output.
- Choose one low-consequence recurring work task and write a source contract before connecting any business data.
- Measure the current manual baseline, then pilot a draft-only Cowork workflow with a named reviewer.
- Convert successful prompt changes into a versioned playbook and project instructions; record why each change was made.
- Review current Anthropic guidance before enabling new connections, computer use, remote execution, or scheduled operation.

Glossary

- **Agentic loop** — The repeated cycle of planning, acting through tools, observing results, correcting, and verifying progress toward an outcome.
- **Approval gate** — A defined pause where a named person reviews specified evidence and authorises the next action.
- **Connector** — A structured integration that lets Claude access a permitted external service through defined capabilities.
- **Connected folder** — A local directory deliberately made available to a Cowork session or project.
- **Control total** — An independently known value used to confirm that a calculation or transformation preserved the expected data.
- **Cowork project** — A persistent workspace with its own files, links, instructions, scheduled tasks, and memory.
- **Dependency gate** — A merge point where upstream work is reconciled before a dependent stage begins.

- **Evidence** — An observable artifact such as a source reference, calculation, preview, log, manifest, or file comparison that supports a claim.
- **Human-in-the-loop** — A workflow in which a person retains responsibility for a defined review or decision step.
- **Manifest** — A structured inventory of files or records and their relevant metadata.
- **Parallel workstream** — A bounded stage that can progress independently because it does not require the intermediate output of another concurrent stage.
- **Playbook** — A reusable operational description of triggers, parameters, sources, stages, outputs, checks, gates, exceptions, and ownership.
- **Project instructions** — Durable context and rules applied across tasks in one Cework project.
- **Provenance** — The traceable origin of a fact, value, quotation, or conclusion.
- **Source contract** — The required inputs, approved scope, freshness, ownership, fallback, and stop conditions for a workflow.
- **Stop condition** — A situation that should halt the workflow and return control to a person.
- **Task contract** — The outcome, context, deliverables, constraints, approvals, evidence, and finish line agreed for one task.
- **Working copy** — A separate destination used for transformations so original sources remain stable and recoverable.